

Plan testów offline: Q11 z modułem Shelly

Co jeszcze trzeba sprawdzić, żeby potwierdzić, że bez chmury i bez internetu Q11 na Shelly umie co najmniej to, co Q11 na Tuya, że da się nią sterować prosto z telefonu, i że po zaniku zasilania pamięta to, co powinna · AMPERE POINT · 28 września 2026 · oprac. Dawid Cekała

Co zmieniło się w v2

Miejsce	Zmiana
Rozdział 1, nowy	Poziomy łączności: co znaczy „z chmurą”, „bez chmury”, „bez internetu”, „bez Wi-Fi”; każdy test przypisany do poziomu
Rozdział 2, nowy	Słowniczek: menu ładowarki, okno godzin, trzy rodzaje harmonogramu, API, webhook, zapamiętywanie pól
Porównanie z Tuya	Tuya lokalnie działa kluczem lokalnym przez Home Assistant; harmonogramu Tuya nie wykonuje telefon, a czy chmura, czy moduł Tuya, rozstrzyga nowy test porównawczy T0
Przygotowanie	Prawdziwy brak internetu dla modułu: stały adres bez bramy; wcześniejsze próby były bez chmury, ale z internetem
T1	Aplikacja Shelly bez internetu to niewiadoma, nie oczekiwany sukces; nowy krok ze stroną WWW przez sieć modułu; hasło sieci modułu
T2	Stan sieci wysyłany do sterownika: co znaczy „4” i kiedy moduł wysyła inny
T3	Okno godzin ma pełne godziny, więc test przez serwer czasu z przesunięciem; zapis z modułu wymaga nowych pól, przeniesiony do osobnego etapu
T4	Przed webhookiem sprawdzenie, jakie zdarzenia moduł obsługuje
T5	Przewidywanie z kodu skryptu, na co patrzeć w pierwszych sekundach; nowy krok z restartem samego modułu w trakcie ładowania; energia sesji zawsze 0
T6	Zapasowa droga odzyskania Wi-Fi
T7, nowy	API: hasło, wartości spoza zakresu, wychodzący WebSocket, połączenia naraz, kopia konfiguracji

W skrócie

- **Cel dla użytkownika.** Zwykły użytkownik, bez Home Assistant, ma prosto sterować ładowarką bez chmury: z aplikacji przez Bluetooth albo ze strony modułu w sieci lokalnej.
- **Co Tuya daje bez chmury.** Większość funkcji, które działają bez internetu, siedzi w sterowniku Q11: menu, przyciski, start po podłączeniu auta, limit prądu, okno godzin, limit energii. Sterowanie z daleka idzie przez chmurę Tuya. Harmonogramy z aplikacji zapisuje chmura; kto je wykonuje, chmura czy moduł Tuya, jeszcze nie wiadomo. Lokalnie Tuya da się sterować kluczem lokalnym przez Home Assistant, ale to droga dla zaawansowanych. Sama aplikacja Tuya, gdy telefon jest w tej samej sieci co ładowarka, też wysyła polecenia lokalnie. Nie sprawdzaliśmy, czy robi to także bez internetu, skoro do logowania potrzebuje chmury.
- **Co sprawdzamy.** Trzy rzeczy. Czy funkcje sterownika działają z modułem Shelly bez internetu. Czy to, co moduł Shelly daje od siebie bez chmury, naprawdę działa: Bluetooth, strona WWW, harmonogramy w module, webhooki, API. I co się dzieje po zaniku zasilania, kiedy dwa układy z własną pamięcią startują naraz.
- **Kolejność od bezpiecznych do ryzykownych.** Reset sieci z menu ładowarki zostawiamy na koniec, bo może skasować Wi-Fi modułu.

1 Poziomy łączności

„Offline” może znaczyć kilka różnych rzeczy. W tym planie rozróżniamy pięć poziomów:

Poziom	Co działa	Czego nie ma	Przykład z życia
A. Z chmurą	Wi-Fi, internet, chmura producenta	—	zwykła praca; aplikacja z dowolnego miejsca
B. Bez chmury	Wi-Fi i internet	chmura producenta wyłączona w module	nasze próby od 25.09
C. Bez internetu	Wi-Fi i sieć lokalna	internet	awaria łącza; firma, która nie wpuszcza ładowarek do internetu
D. Bez Wi-Fi	Bluetooth albo własna sieć modułu, telefon obok ładowarki	router	garaż bez zasięgu Wi-Fi
E. Bez niczego	sama ładowarka	żadnej łączności	tylko menu i przyciski

„Offline” w tym planie oznacza poziomy C, D i E. Na poziomie B wszystko już sprawdziliśmy (HTTP, WebSocket, MQTT, OCPP przez most). Brak internetu symulowaliśmy tylko dla serwera czasu.

Test	Poziom
T1 Bluetooth, strona WWW przez sieć modułu	D, oraz C dla porównania
T2 funkcje sterownika	C i E
T3 okno godzin, tryb, limit energii	C i E
T4 harmonogram, webhook, pamięć w module	C
T5 zanik zasilania	C
T6 reset sieci z menu	E, skutek dla modułu
T7 API	C

Słowniczek

- **Menu ładowarki.** Ustawienia na samej Q11: wyświetlacz i dwa przyciski na obudowie, 5 języków. Działa bez żadnego modułu. W planie „z menu” znaczy: ktoś stoi przy ładowarce i ustawia przyciskami.
- **Okno godzin ładowania.** Ustawienie sterownika Q11: godzina od i godzina do, w których wolno ładować, np. 22–6 dla nocnej taryfy. Działa w trybie „harmonogram”. Sterownik pilnuje okna sam, potrzebuje tylko aktualnej godziny od modułu. Zapisane w dwóch bajtach: godzina startu i godzina końca, **bez minut**.
- **Harmonogram w aplikacji Tuya.** Wpisy tworzy aplikacja, a przechowuje chmura. Telefon do wykonania nie jest potrzebny. Kto wykonuje wpis, nie jest rozstrzygnięte, bo źródła są sprzeczne. Za chmurą przemawia definicja produktu Q21 w Tuya, w której stoi *CloudTiming*: „cloud timing without local timing”. Za modulem Tuya przemawiają logi ładowarki testowej (Q11 PRO): wykonanie wpisu widać tylko jako raport samej ładowarki, bez polecenia z chmury, które pojawia się przy scenach. Rozstrzyga test T0b.
- **Harmonogram w module Shelly.** Do 20 zadań zapisanych w samym module, np. „o 22:00 ustaw 16 A”. Moduł wykonuje je sam, bez chmury i internetu. Potrzebuje godziny, więc przy braku internetu wymaga lokalnego serwera czasu.

- **API.** Zestaw poleceń, którymi inne programy rozmawiają z modułem, np. „podaj stan” albo „ustaw limit 10 A”. Te same polecenia idą różnymi drogami: HTTP, WebSocket, MQTT, Bluetooth.
- **Webhook.** Adres WWW, który moduł **sam** wywołuje, gdy coś się stanie, np. „zaczęło się ładowanie”. To odwrotność API: przy API program pyta moduł, przy webhooku moduł sam zawiadamia program. Przykład: moduł wywołuje `http://192.168.0.57:8080/zdarzenie`, a program na laptopie zapisuje zdarzenie.
- **Zapamiętywanie pola.** Ustawienie pola w portalu Shelly („persisted”): moduł trzyma wartość w swojej pamięci i po restarcie ją odtwarza. U nas zapamiętywane są limit prądu i włącznik ładowania.
- **Sieć modułu.** Moduł może sam wystawić sieć Wi-Fi (punkt dostępowy), do której łączy się telefon, bez routera. **Stacja Wi-Fi** to druga strona: moduł jako klient sieci biura.

2 Co ma Q11 na Tuya bez chmury, a co Q11 na Shelly

Funkcja	Gdzie siedzi	Q11 na Tuya	Q11 na Shelly	Test
Start ładowania po podłączeniu auta	sterownik	✓	✓ sprawdzone 25.09	T2
Limit prądu z menu ładowarki	sterownik	✓	? sprawdzany tylko z modułu, nie z menu	T2
Okno godzin ładowania	sterownik	✓ z menu; z aplikacji przez chmurę	?	T3
Limit energii sesji	sterownik	✓ z menu	? sterownik nigdy nie zgłosił tego punktu	T3
Tryb pracy: natychmiast, energia, harmonogram	sterownik	✓	?	T3
Godzina dla sterownika	moduł	z chmury Tuya; bez internetu brak	✓ z lokalnego serwera czasu, sprawdzone	T2
Ikona sieci na wyświetlaczu	moduł	odbija stan sieci i chmury	🟡 moduł wysyła „4” (chmura) także przy wyłączonej chmurze	T2
Sterowanie z komputera w sieci lokalnej	moduł	🟡 kluczem lokalnym przez Home Assistant; nie dla zwykłego użytkownika	✓ HTTP, WebSocket, MQTT, bez klucza, sprawdzone	—
Sterowanie z telefonu bez internetu	moduł	🟡 aplikacja Tuya w tej samej sieci wysyła polecenia lokalnie (obserwacja 07.09); bez internetu nie sprawdzano, T0a	? aplikacja Shelly przez Bluetooth; strona WWW modułu	T1
Harmonogram z aplikacji	chmura albo moduł	🟡 telefon niepotrzebny; kto wykonuje, chmura czy moduł Tuya, rozstrzyga T0b	? do 20 zadań w module	T4
Powiadomienie o zdarzeniu do własnego systemu	moduł	✗	? webhooki	T4

Funkcja	Gdzie siedzi	Q11 na Tuya	Q11 na Shelly	Test
Pamięć ustawień po zaniku zasilania	sterownik i moduł	sterownik pamięta swoje	? dwa układy z pamięcią	T5
Reset sieci z menu ładowarki	sterownik → moduł	✓ moduł Tuya wchodzi w parowanie	?	T6

Porównanie z Tuya, opcjonalnie (T0). Dwie niewiadome po stronie Tuya da się rozstrzygnąć tylko na Q11 z modułem Tuya odciętym od internetu. T0a: czy aplikacja Tuya steruje przez sieć lokalną. T0b: czy wpis harmonogramu wykona się bez internetu; jeśli tak, wykonuje go moduł Tuya, a nie chmura. Odcięcie jednej ładowarki od internetu wymaga reguły w routerze, którą ustawia administrator sieci. Robimy to tylko wtedy, jeśli będzie taka możliwość. Sam telefon bez sieci niczego tu nie rozstrzyga, bo ładowarka nadal ma internet.

3 Założenia i przygotowanie

- Chmura Shelly wyłączona przez cały czas, jak dotąd.
- W sieci działają kontenery: broker MQTT i lokalny serwer czasu (192.168.0.57). Serwer czasu musi działać **przed** każdym włączeniem ładowarki, bo moduł pyta o czas zaraz po starcie.
- Monitor logu przez Wi-Fi uruchomiony przed każdym testem. Przy zaniku zasilania monitor sam się wznawia po powrocie modułu.
- Symulator auta podłączony: stany „podłączone” (9 V) i „ładuje” (6 V). Bez obciążenia prądowego, więc prąd i moc faz zostają poza tym planem.
- Laptop ma kartę Bluetooth (Intel). Do testów z komputera służy biblioteka b1eak w Pythonie. Telefon z aplikacją Shelly jest potrzebny do testu z użytkownikiem.
- Wi-Fi laptopa i router zostają bez zmian.
- **Moduł na poziomie C.** Moduł dostaje stały adres (ten sam, 192.168.0.238) bez bramy do internetu. Widzi laptopa, broker i serwer czasu, ale nie internet. Zmiana jest odwracalna z sieci lokalnej: powrót na adres automatyczny. Przed zmianą sprawdzamy, że sieć modułu jest włączona, bo to droga ratunkowa, gdyby moduł zniknął z sieci.
- Po każdym teście zapis w tabeli wyników: ✓ działa, 🟡 działa z zastrzeżeniem, ✗ nie działa, z opisem tego, co zaobserwowano.

4 Testy

4.1 T1. Bluetooth i strona WWW

Cel. Potwierdzić, że ładowarką da się sterować bez Wi-Fi i bez internetu, z telefonu i z komputera.

Krok	Co robimy	Wynik oczekiwany
1a	Skan Bluetooth z laptopa	moduł widoczny pod swoją nazwą, z reklamą Shelly
1b	Polecenie przez Bluetooth z laptopa: odczyt stanu, potem zmiana limitu prądu na 10 A	sterownik potwierdza w logu w ciągu 1 s, tak jak przy Wi-Fi

Krok	Co robimy	Wynik oczekiwany
1c	Telefon z wyłączonym Wi-Fi i danymi komórkowymi, aplikacja Shelly przez Bluetooth: włącz/wyłącz ładowanie, zmiana limitu	do ustalenia. Aplikacja wymaga konta w chmurze, a chmura modułu jest wyłączona; może nie pokazać urządzenia. Zapisujemy, co widać i ile trwa połączenie
1f	Telefon łączy się z siecią modułu, przeglądarka otwiera stronę modułu, zmiana limitu i włącznika	działa bez routera i bez aplikacji; 24.09 działało na symulatorze, teraz sprawdzamy na produkcie Q11
1d	Moduł bez Wi-Fi: przez Bluetooth wyłączamy stację Wi-Fi modułu. Wcześniej sprawdzamy, że sieć modułu jest włączona jako droga awaryjna, i czy ma hasło . Powtarzamy 1b, 1c i 1f	działa bez Wi-Fi; log niedostępny, więc dowodem są wyświetlacz ładowarki i odpowiedź przez Bluetooth
1e	Przez Bluetooth włączamy stację Wi-Fi z powrotem	moduł wraca do sieci biura w ciągu minuty, log wraca. Jeśli moduł zażąda hasła sieci, wpisuje je użytkownik

Ryzyko. Jeśli w 1d Bluetooth zawiedzie, moduł zostaje bez Wi-Fi. Droga powrotu: sieć modułu z telefonu. Dlatego 1d dopiero po udanych 1b i 1f.

Wniosek do analizy GAP, niezależnie od wyniku. Jeśli sieć modułu nie ma hasła, każdy w pobliżu może sterować ładowarką przez stronę WWW.

Narzędzie do napisania. `narzedzia\ble_rpc.py`: klient poleceń Shelly przez Bluetooth. Ta sama treść JSON co przez Wi-Fi, opakowana w ramkę Bluetooth Shelly.

4.2 T2. Funkcje sterownika bez internetu

Cel. Potwierdzić, że sterownik robi swoje, gdy moduł ma tylko sieć lokalną (poziom C).

Krok	Co robimy	Wynik oczekiwany
2a	Symulator: wolne → podłączone → łąduje	sterownik przechodzi stany „wolna” → „auto podłączone” → „łąduje”, zamyka stycznik; moduł pokazuje to w polach
2b	Limit prądu z menu ładowarki : 8 A	sterownik sam zgłasza 8 A; pole w module aktualizuje się bez naszego zapisu
2c	Limit prądu z modułu przy ładowaniu: 12 A	sterownik potwierdza w 1 s; wyświetlacz pokazuje 12 A
2d	Godzina po starcie: włączamy ładowarkę, patrzymy na pierwsze ramki czasu	przez około 20 s moduł odpowiada zerową datą, potem poprawną; sprawdzamy, co w tym czasie pokazuje wyświetlacz

Krok	Co robimy	Wynik oczekiwany
2e	Stan sieci wysyłany do sterownika: obserwacja przy poziomie C, a potem w T1d przy wyłączonej stacji Wi-Fi	według opisu protokołu Tuya: 2 = brak routera, 3 = router bez chmury, 4 = połączony z chmurą. Moduł wysyła „4” także bez chmury. Zapisujemy, czy kiedykolwiek wysyła 2 albo 3. Od tego zależy, czy ikona na wyświetlaczu coś znaczy

4.3 T3. Okno godzin, tryb pracy, limit energii

Cel. Trzy funkcje, które Tuya obsługuje z aplikacji, a my na razie tylko z menu. Ustalić, czy sterownik je zgłasza i wykonuje, oraz co trzeba dodać w produkcie Shelly.

Narzędzie do napisania: serwer czasu z przesunięciem. narzędzia\zegar_przesuniety.py podaje czas przesunięty tak, żeby za 2 minuty wybiła pełna godzina. Okno godzin ma tylko pełne godziny, więc bez tego test trwałby do dwóch godzin. Moduł przekazuje sterownikowi tylko to, co dostał od serwera czasu, więc przesuujemy zegar ładowarki bez ruszania zegara laptopa. Na czas testu zatrzymujemy kontener z serwerem czasu (port 123), a po zmianie czasu restartujemy sam moduł, żeby od razu pobrał nowy.

Krok	Co robimy	Wynik oczekiwany
3a	Serwer z przesunięciem: za 2 min wybije godzina G. Z menu: tryb „harmonogram”, okno od G do G+1. Symulator w stanie „ładuje”. Potem przesunięcie na 2 min przed G+1	sterownik zgłasza tryb „harmonogram” i okno (G, G+1); przed G stan „czeka”, o G ładowanie rusza, o G+1 staje
3b	To samo bez ważnego czasu: serwer czasu zatrzymany, ładowarka włączona na nowo	sterownik dostaje zerową datę; zapisujemy, co robi z oknem: nie startuje, startuje od razu albo coś innego. Potem serwer czasu z powrotem
3c	Z menu: tryb „energia”, limit 1 kWh	sterownik zgłasza tryb „energia” i, oby, limit energii; bez obciążenia licznik nie ruszy, więc sprawdzamy tylko zgłaszanie
3d	Z menu: sprawdzamy, czy da się ustawić okno z minutami	jeśli tak, założenie „tylko pełne godziny” jest błędne i trzeba je poprawić w zestawieniu punktów danych

Zapis tych trzech ustawień z modułu to osobny etap. Moduł nie ma polecenia do zapisu dowolnego punktu danych. Zapisuje tylko skrypt usługi, i tylko do pól zdefiniowanych w produkcie. Trzeba więc dodać pola w portalu i wgrać nową konfigurację, a to kasuje Wi-Fi modułu.

Wynik uboczny. Lista pól do dodania w portalu: tryb pracy, okno godzin, limit energii, błędy, sygnał z auta, temperatura, wersja sterownika, energia ostatniej sesji.

4.4 T4. Harmonogram, webhook i pamięć w module

Cel. To, co moduł Shelly daje ponad Tuya, ma działać bez chmury i przetrwać restart.

Krok	Co robimy	Wynik oczekiwany
4a	Harmonogram w module: o +2 min ustaw limit 6 A, o +4 min 16 A	oba zadania odpalają o czasie, sterownik potwierdza
4b	Najpierw lista zdarzeń, na które moduł umie wywołać webhook (polecenie, które nic nie zmienia)	wiemy, czy zmiana naszych pól (stan, limit, włącznik) jest zdarzeniem, czy tylko zdarzenia systemowe
4c	Webhook: przy zmianie stanu ładowania moduł wywołuje adres na laptopie (mały nasłuch HTTP w Pythonie)	wywołanie przy każdej zmianie, z treścią zdarzenia; jeśli 4b pokaże brak takiego zdarzenia, zapisujemy X i powód
4d	Pamięć klucz-wartość: zapis wartości testowej	odczyt daje to samo; po T5 sprawdzamy, czy przetrwała zanik zasilania
4e	Harmonogram i webhook po restarcie modułu (samo polecenie restartu, bez zaniku zasilania)	nadal są i działają

4.5 T5. Zanik zasilania i restart modułu

Cel. Rozstrzygnąć, co pamięta sterownik, co moduł i kto komu narzuca wartość po starcie.

Co przewiduje kod naszego skryptu. Po starcie skrypt czyta limit i włącznik ze sterownika i nadpisuje nimi pola w module. Do sterownika pisze tylko wtedy, gdy pole się zmieni na wartość inną niż ostatnio otrzymana ze sterownika. Normalnie wygrywa więc sterownik.

Słabe miejsce: zanim sterownik pierwszy raz zgłosi limit, skrypt nie zna jego wartości. Każda zmiana pola w tych kilku sekundach pójdzie do sterownika. Taką zmianą może być odtworzenie zapamiętanej wartości, harmonogram albo polecenie z aplikacji. Dlatego w 5d patrzymy na pierwsze sekundy logu.

Krok	Co robimy	Wynik oczekiwany / co zapisujemy
5a	Stan wyjściowy, zapisany z logu: limit 10 A ustawiony z modułu , ładowanie wyłączone, symulator „podłączone”; w drugiej rundzie dodatkowo tryb „harmonogram” z oknem, limit energii 1 kWh, wpis klucz-wartość i harmonogram z T4	pełny rzut wszystkich punktów danych i pól modułu
5b	Wyłącznik ładowarki: wyłączyć na 30 s, włączyć	moduł i sterownik startują; monitor wraca
5c	Zrzut po starcie (odpowiedź sterownika na zapytanie o wszystkie punkty) porównany z 5a	które wartości sterownik zachował; oczekujemy limitu prądu, trybu, okna i limitu energii; zapisujemy każdą różnicę

Krok	Co robimy	Wynik oczekiwany / co zapisujemy
5d	Pierwsze sekundy logu po starcie	czy moduł wysłał zapis limitu albo włącznika, zanim sterownik zgłosił swoje wartości. Jeśli tak, to jest scenariusz z przykładu poniżej
5e	Włącznik ładowania: przed wyłączeniem „włączone”, symulator „ładuje”	czy po powrocie zasilania ładowanie rusza samo, czy czeka na polecenie; to zachowanie sterownika, ważne dla klienta
5f	Czas: po ilu sekundach od startu sterownik dostał poprawną godzinę	z logu; wcześniej zerowa data
5g	Moduł: konfiguracja Wi-Fi, MQTT, serwer czasu, harmonogram, webhook, wpis klucz-wartość	wszystko na miejscu
5h	Energia i czas sesji w polach modułu	energia sesji zawsze 0 , bo liczymy ją z licznika całkowitego, którego sterownik nie zgłasza; czas sesji 0, bo trzymany tylko w pamięci RAM
5i	Powtórka 5a–5d z limitem ustawionym z menu ładowarki	drugi kierunek: czy moduł po starcie nie nadpisze ustawienia z menu swoją zapamiętaną wartością
5j	Restart samego modułu w trakcie ładowania (symulator „ładuje”), sterownik pracuje dalej	czy sterownik przerywa ładowanie, gdy moduł przestaje dawać znak życia; co moduł wysyła po powrocie. Tak będzie przy każdej aktualizacji oprogramowania modułu

Dlaczego to ważne. Klient ustawia z menu 8 A, bo ma słabe zabezpieczenie. Moduł pamięta z aplikacji 16 A. Po zaniku zasilania moduł wstaje, wysyła 16 A i ładowarka rusza z 16 A. Taki błąd byłby groźny, więc 5d i 5i to najważniejsze punkty planu.

Rekomendacja, jeśli 5c pokaże, że sterownik sam pamięta limit i włącznik. Wyłączyć w portalu zapamiętywanie tych dwóch pól. Wtedy nic nie dają, bo sterownik i tak zgłasza swoje wartości po starcie, a są jedynym źródłem opisanego ryzyka.

4.6 T6. Reset sieci z menu ładowarki

Cel. W Q11 na Tuya pozycja menu „reset sieci” wprowadza moduł w parowanie. Sprawdzić, co to polecenie sterownika robi z modułem Shelly.

Krok	Co robimy	Wynik oczekiwany
6a	Monitor na najwyższym poziomie logu; z menu ładowarki: reset sieci	w logu ramka od sterownika z poleceniem resetu Wi-Fi i reakcja modułu: ignoruje, kasuje Wi-Fi albo wchodzi w parowanie
6b	Jeśli moduł skasował Wi-Fi: ponowne dodanie do sieci	moduł wraca do sieci

Drogi powrotu w 6b, w tej kolejności: (1) parowanie z telefonu przez Bluetooth w aplikacji Shelly; 24.09 zawiodło, bo aplikacja zapisała nazwę sieci ze spacją na końcu. (2) Sieć modułu i jego strona WWW: nazwa sieci i hasło wpisane przez użytkownika. USB odpada, bo moduł jest zasilany z płytki.

Ryzyko. Utrata Wi-Fi modułu, dlatego ten test jest na końcu. Wynik zapisujemy jako fakt do dokumentacji dla fabryki.

4.7 T7. API

Cel. Zobaczyć, jak działa API modułu i gdzie są jego granice. W module jest 166 poleceń w 30 grupach. Sprawdzamy te, które mają znaczenie dla klienta i dla naszego zaplecza.

Krok	Co robimy	Wynik oczekiwany / co zapisujemy
7a	Wartości spoza zakresu przez API: limit 5, 17, 20 i 32 A; tekst zamiast liczby	kto odrzuca: pole modułu, skrypt czy sterownik; co trafia do sterownika. Definicja Tuya dopuszcza 6–32 A, a Q11 ma 16 A. Z symulatorem bez obciążenia to bezpieczne
7b	Wychodzący WebSocket: moduł sam łączy się z serwerem testowym na laptopie	co moduł wysyła sam; czy serwer może odesłać polecenie, np. limit 10 A, i czy dotrze ono do sterownika. To podstawa tłumacza OCPP na serwerze AmperePoint
7c	Połączenia naraz: most, monitor, strona WWW i kolejne klienty WebSocket	przy którym połączeniu moduł zaczyna odmawiać albo zwalniać
7d	Kopia konfiguracji na laptop	co zawiera; jeśli hasło Wi-Fi, plik nie może trafić do repozytorium
7e	Hasło do API: ustawiamy, sprawdzamy HTTP, WebSocket, Bluetooth i stronę WWW; potem zdejmujemy	które drogi wymagają hasła. Most OCPP i monitor przestaną działać, dopóki nie dopiszemy logowania. Hasło zapisujemy w pliku projektu poza repozytorium, bo zgubione oznacza reset fabryczny
7f	Opcjonalnie: kanał UDP	czy działa i do czego mógłby służyć

5 Kolejność i czas

Kolejność	Test	Poziom	Ryzyko	Czas	Potrzebny użytkownik
1	T1a–c, T1f Bluetooth i strona WWW	D przy działającym Wi-Fi	małe	1 h 15 min z narzędziem	tak, telefon
2	Przejście modułu na poziom C	C	małe	15 min	nie
3	T2 funkcje sterownika	C, E	małe	30 min	tak, menu i symulator

Kolejność	Test	Poziom	Ryzyko	Czas	Potrzebny użytkownik
4	T5 zanik zasilania, pierwsza runda, z 5j	C	małe	1 h	tak, wyłącznik
5	T3 okno, tryb, limit energii	C, E	małe	1 h z narzędziem	tak, menu
6	T4 harmonogram, webhook, pamięć	C	małe	45 min	nie
7	T7a–d API	C	małe	1 h	nie
8	T5 druga runda, po T3 i T4	C	małe	30 min	tak
9	T7e hasło do API	C	średnie	20 min	nie
10	T1d–e Bluetooth bez Wi-Fi	D	średnie	30 min	tak, telefon jako droga awaryjna
11	T6 reset sieci z menu	E	duże	20 min plus ewentualne dodanie do sieci	tak
12	Powrót modułu na adres automatyczny, poziom B	B	małe	10 min	nie

Razem około 7,5 godziny, czyli dwa dni pracy. Po wszystkim: tabela wyników do analizy GAP v4 i uzupełnienie dokumentacji dla fabryki o wyniki T5 i T6.

6 Czego ten plan nie obejmuje

- Prąd i moc faz pod obciążeniem, pełna sesja z energią: wymaga obciążenia, osobny test.
- Zapis trybu, okna godzin i limitu energii z modułu: wymaga nowych pól w produkcie, osobny etap po T3.
- Aktualizacja sterownika przez moduł: wymaga pliku od fabryki.
- Ekran ładowarki w aplikacji Shelly: sprawa portalu, nie łączności.
- Porównanie z Tuya bez internetu (T0): tylko jeśli administrator sieci odetnie jedną ładowarkę od internetu.